Assignment1 - Probability, Linear Algebra and Computational Programming

Name: Edward Huang
NetID: yh475
Collaborators: N/A

Exercise1 - Probabilistic Reasoning

1.1

Because the first dice must be 1, the remaining dices must sum to 9 in order for the total to be 10.
The combination of the last 2 dices will be (3,6), (4,5), (5,4), (6,3) 4 combinations.

Therefore, the probability is
$P(\text{sum} = 10 \mid \text{first die} = 1) = \frac{4}{36} = \frac{1}{9}$.

1.2

In [1]:
```
import numpy as np

n = 1000000
res = 0
for i in range(n):
    dice2 = np.random.randint(1,7)
    dice3 = np.random.randint(1,7)
    if(dice2 + dice3) == 9: res +=1
print("The fraction is approaching: ", res/n)
```

```
The fraction is approaching:  0.111038
```

1.3

Let D denote the event that a person has the disease and T denote the event that the test is positive.

We are given:

$P(D) = 0.001, \quad P(\neg D) = 0.999$
$P(T \mid D) = 0.95, \quad P(T \mid \neg D) = 0.03$

By Bayes' theorem
$P(D \mid T) = \frac{0.95 \times 0.001}{0.95 \times 0.001 + 0.03 \times 0.999} \approx 0.0307$.

Therefore, the probability that a person who tests positive actually has the disease is approximately 3.1%.

1.4

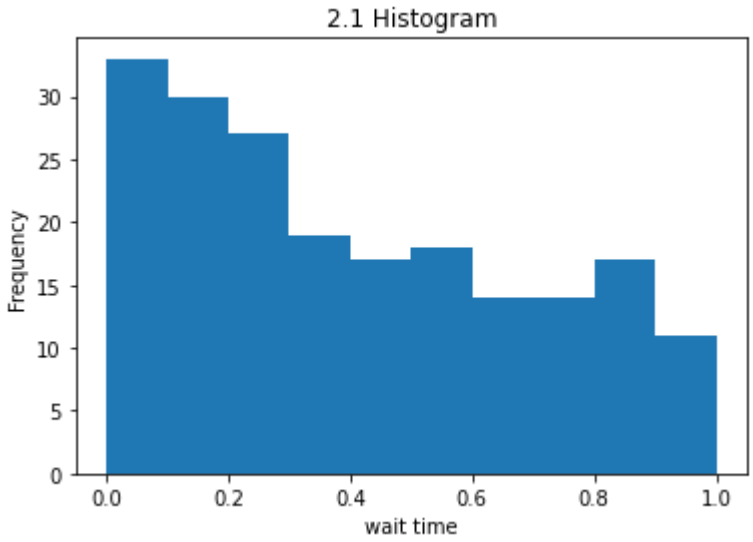
The exepcted value is

$$\mathbb{E}[X] = (-1)(0.2) + (0)(0.5) + (1)(0.3) = -0.2 + 0 + 0.3 = 0.1$$

Exercise2 - Probability Distributions and Modeling

2.1

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt

data = np.loadtxt("data/wait_times.csv")
plt.hist(data, bins = 10, range=[0,1])
plt.xlabel("wait time")
plt.ylabel("Frequency")
plt.title("2.1 Histogram")
plt.show()
```



2.2

In [3]:
```
mean = np.mean(data)
var = np.var(data)
std = np.std(data)
mean_hour = mean * 8
std_hour = std * 8

print("The mean: ", mean)
print("The variance: ", var)
print("The standard deviation: ", std)
print("The mean(in hours): ", mean_hour)
print("The standard deviation(in hours): ", std_hour)
```

```
The mean:  0.40419623697994356
The variance:  0.08154808395295482
The standard deviation:  0.28556625142504993
The mean(in hours):  3.2335698958395485
The standard deviation(in hours):  2.2845300114003995
```

2.3

If f is a valid probability density funciton, fx(x) must integrate to 1.
Given
$f_X(x) = \alpha e^{-x}$ for $0 \le x \le 1$ and $f_X(x) = 0$

otherwise, we require
$\int_0^1 \alpha e^{-x}\, dx = 1.$

Evaluating the integral,
$\int_0^1 e^{-x}\, dx = \left[-e^{-x}\right]_0^1 = 1 - e^{-1}.$

Thus,
$\alpha(1 - e^{-1}) = 1,$
which gives
$\alpha = \frac{1}{1-e^{-1}}.$

Therefore, the exact value of α is
$\alpha = \frac{1}{1-e^{-1}}.$

In [4]:
```
alpha = 1 / (1 - np.exp(-1))
print(f"The alpha is: {alpha:.3f}")
```

```
The alpha is: 1.582
```

2.4

The CDF is defined as
$F_X(x) = P(X < x)$.

The probability density function is
$f_X(x) = \alpha e^{-x}$ for $0 \le x \le 1$ otherwise 0.

For $x < 0$:
$F_X(x) = 0$.

For $0 \le x \le 1$:
$F_X(x) = \int_0^x \alpha e^{-t}\, dt. = \alpha\left[-e^{-t}\right]_0^x = \alpha(1 - e^{-x})$.

For $x > 1$:
$F_X(x) = \int_0^1 \alpha e^{-t}\, dt = \alpha(1 - e^{-1}) = 1$.

Thus, the CDF is $F_X(x) = \begin{cases} 0, & x < 0 \\ \alpha(1 - e^{-x}), & 0 \le x \le 1 \\ 1, & x > 1 \end{cases}$, where $\alpha = \dfrac{1}{1 - e^{-1}}$.

2.5

Given
$f_X(x) = \alpha e^{-x}$ for $0 \le x \le 1$ and 0 otherwise,
the expected value is
$E_X[X] = \int_0^1 x\alpha e^{-x}\, dx$.

Factoring out α gives
$\alpha \int_0^1 x e^{-x}\, dx$.

Using the provided formula
$\int x e^{-x}\, dx = -e^{-x}(x + 1)$,
we evaluate the integral as
$\int_0^1 x e^{-x}\, dx = \left[-e^{-x}(x + 1)\right]_0^1. = -e^{-1}(1 + 1) - \left(-e^{0}(0 + 1)\right)$
$= -2e^{-1} + 1$
$= 1 - 2e^{-1}$.

Thus, $E_X[X] = \alpha(1 - 2e^{-1})$.

In [5]:
```
EX = alpha * (1 - 2 * np.exp(-1))
print(f"E_X[X] ≈ {EX:.3f}")
print(f"Expected waiting time ≈ {8 * EX:.3f} hours")
```

```
E_X[X] ≈ 0.418
Expected waiting time ≈ 3.344 hours
```

2.6

For the continuous random variable,
$Var(X) = E[X^2] - (E[X])^2$.

Using the provided formula
$\int x^2 e^{-x}\, dx = -e^{-x}(x^2 + 2x + 2)$,
we compute $E[X^2] = \int_0^1 x^2 \alpha e^{-x}\, dx$.

Factoring out α gives
$E[X^2] = \alpha \int_0^1 x^2 e^{-x}\, dx$.

Applying the bounds,
$\int_0^1 x^2 e^{-x}\, dx = \left[-e^{-x}(x^2 + 2x + 2)\right]_0^1$
$= -e^{-1}(1^2 + 2 \cdot 1 + 2) - \left(-e^{0}(0^2 + 0 + 2)\right)$
$= -5e^{-1} + 2$.

Thus,
$E[X^2] = \alpha(2 - 5e^{-1})$.

The variance is therefore $Var(X) = \alpha(2 - 5e^{-1}) - \left(\alpha(1 - 2e^{-1})\right)^2$.

In [6]:
```
EX2 = alpha * (2 - 5 * np.exp(-1))
varX = EX2 - EX**2
stdX = np.sqrt(varX)
std_hours = 8 * stdX
print(f"Var(X) ≈ {varX:.3g}")
print(f"Std(X) ≈ {stdX:.3g}")
print(f"Std(X) ≈ {std_hours:.3g} hours")
```

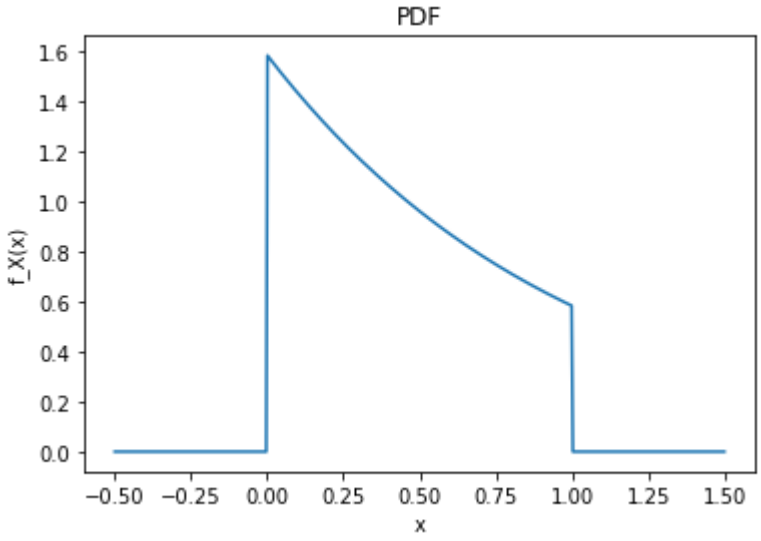
```
Var(X) ≈ 0.0793
Std(X) ≈ 0.282
Std(X) ≈ 2.25 hours
```

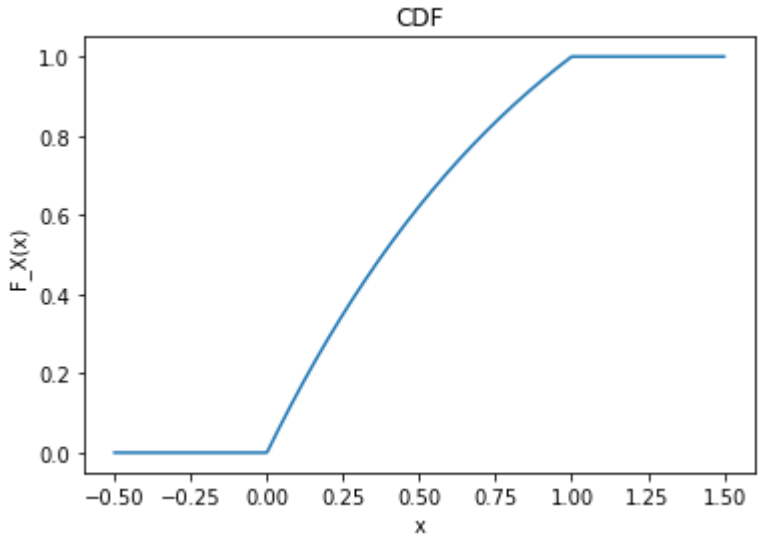
2.7

Plotting functions which x from -0.5 to 1.5

In [7]:
```
def pdf(x):
    if 0 <= x <= 1:
        return alpha * np.exp(-x)
    else:
        return 0

def cdf(x):
    if x < 0:
        return 0
    elif x <= 1:
        return alpha * (1 - np.exp(-x))
    else:
        return 1
```

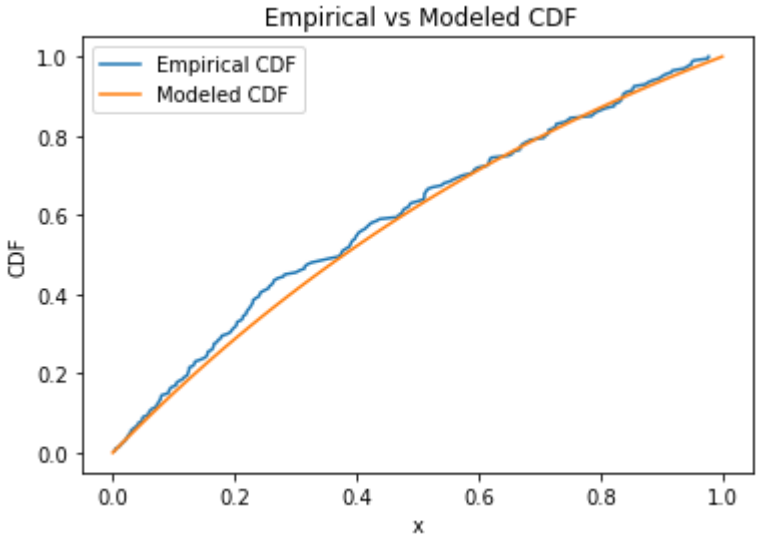
In [8]:
```
x = np.linspace(-0.5, 1.5, 500)

pdf_vals = [pdf(xi) for xi in x]
cdf_vals = [cdf(xi) for xi in x]

# plot PDF
plt.figure()
plt.plot(x, pdf_vals)
plt.xlabel("x")
plt.ylabel("f_X(x)")
plt.title("PDF")
plt.show()

# plot CDF
plt.figure()
plt.plot(x, cdf_vals)
plt.xlabel("x")
plt.ylabel("F_X(x)")
plt.title("CDF")
plt.show()
```





2.8

In [9]:
```
empirical_x = np.sort(data)
n = len(empirical_x)
empirical_y = np.arange(1, n + 1)/n
x_model = np.linspace(0,1)
y_model = [cdf(xi) for xi in x_model]

plt.figure()
plt.plot(empirical_x, empirical_y, label = "Empirical CDF")
plt.plot(x_model, y_model, label = "Modeled CDF")
plt.xlabel("x")
plt.ylabel("CDF")
plt.title(" Empirical vs Modeled CDF")
plt.legend()
plt.show()
```



2.9

For the CDF,
$F_X(x) = \alpha(1 - e^{-x})$, for $0 \le x \le 1$
$$\alpha = \frac{1}{1 - e^{-1}}$$

let u = Fx(x)
$u = \alpha(1 - e^{-x})$
$\Rightarrow \frac{u}{\alpha} = 1 - e^{-x}$
$\Rightarrow e^{-x} = 1 - \frac{u}{\alpha}$
and the x will be
$$x = -\ln\left(1 - \frac{u}{\alpha}\right)$$

$$F_X^{-1}(u) = -\ln\left(1 - \tfrac{u}{\alpha}\right), \quad 0 \le u \le 1$$

In [10]:
```
#Build inverse cdf function
def inverse_cdf(u):
    return -np.log(1-u/alpha)
```

2.10

In [11]:
```
#Generate synthetic data
import pandas as pd
u = np.random.rand(10000)
synthetic_data = inverse_cdf(u)

theory_mean = alpha * (1 - 2 * np.exp(-1))
theory_std = np.sqrt(alpha * (2 - 5 * np.exp(-1)) - theory_mean**2)
syn_mean = np.mean(synthetic_data)
syn_std = np.std(synthetic_data)

comparison_table = pd.DataFrame({
    "Mean": [mean, theory_mean, syn_mean],
    "Standard Deviation": [std, theory_std, syn_std]
}, index=["Empirical", "Theoretical", "Synthetic"])

comparison_table
```

Out[11]:

	Mean	Standard Deviation
Empirical	0.404196	0.285566
Theoretical	0.418023	0.281649
Synthetic	0.417426	0.280799

2.11

In [12]:
```
from scipy.stats import kstest


model_cdf_vec = np.vectorize(cdf)
ks_model = kstest(data, model_cdf_vec)
ks_uniform = kstest(data, 'uniform')

print("KS test vs modeled distribution:")
print("Statistic:", ks_model.statistic)
print("p-value :", ks_model.pvalue)

print("\nKS test vs uniform distribution:")
print("Statistic:", ks_uniform.statistic)
print("p-value :", ks_uniform.pvalue)
```

```
KS test vs modeled distribution:
Statistic: 0.06721686728074683
p-value : 0.3126882015911675

KS test vs uniform distribution:
Statistic: 0.17040498606312987
p-value : 1.5118541753740167e-05
```

The KS test comparing the empirical data to the modeled distribution yields a p-value of 0.313, which is greater than the 0.05 significance level.
Therefore, we fail to reject the null hypothesis and conclude that the modeled distribution provides a reasonable fit to the data.
In contrast, the KS test comparing the empirical data to a uniform distribution results in a p-value of 1.5×10^-5, leading us to reject the null hypothesis.
Which means the empirical data are not uniformly distributed.

2.12

In [13]:
```
synthetic_hours = 8 * synthetic_data

# 1. Wait time is more than 6 hours
pro_6 = np.mean(synthetic_hours > 6)

# 2. Wait time is less than 1 hour
pro_1 = np.mean(synthetic_hours < 1)

# 3. Wait time is less than one additional hour given already waite
d 3 hours
# P(X < 4 | X > 3)
pro_conditional = np.mean(synthetic_hours[synthetic_hours > 3] < 4)

# 4. Wait time is between 3 and 5 hours
pro_3_5 = np.mean((synthetic_hours > 3) & (synthetic_hours < 5))

# 5. 90th percentile
p90 = np.percentile(synthetic_hours, 90)

# 6. 99th percentile
p99 = np.percentile(synthetic_hours, 99)

# print results
print("P(wait > 6 hours):", pro_6)
print("P(wait < 1 hour):", pro_1)
print("P(wait < 4 | wait > 3):", pro_conditional)
print("P(3 < wait < 5 hours):", pro_3_5)
print("90th percentile (hours):", p90)
print("99th percentile (hours):", p99)
```

```
P(wait > 6 hours): 0.1626
P(wait < 1 hour): 0.1866
P(wait < 4 | wait > 3): 0.255327545382794
P(3 < wait < 5 hours): 0.2419
90th percentile (hours): 6.711993895094193
99th percentile (hours): 7.85713035214339
```

Exercise3 - Linear Algebra Operations and Theory

3.1

Exercise 3 - Linear Algebra Operations & Theory

3.1

1. AA : $\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix} = \begin{bmatrix}7 & 10\\ 15 & 22\end{bmatrix}$

2. AA^T : $\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}1 & 3\\ 2 & 4\end{bmatrix} = \begin{bmatrix}5 & 11\\ 11 & 25\end{bmatrix}$

3. Ab : $\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}-1\\ 1\end{bmatrix} = \begin{bmatrix}1\\ 1\end{bmatrix}$

4. Ab^T : Invalid, number of column of A doesn't match the number of row of b^T

5. bA : Invalid, number of column of b doesn't match the number of row of A

6. $b^TA = \begin{bmatrix}-1 & 1\end{bmatrix}\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix} = \begin{bmatrix}2 & 2\end{bmatrix}$

7. bb : Invalid, number of row of b doesn't match the number of row of b

8. $b^Tb = \begin{bmatrix}-1 & 1\end{bmatrix}\begin{bmatrix}-1\\ 1\end{bmatrix} = \begin{bmatrix}2\end{bmatrix}$

9. bb^T : $\begin{bmatrix}-1\\ 1\end{bmatrix}\begin{bmatrix}-1 & 1\end{bmatrix} = \begin{bmatrix}1 & -1\\ -1 & 1\end{bmatrix}$

10. $A \circ A$: $\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix} \circ \begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix} = \begin{bmatrix}1 & 4\\ 9 & 16\end{bmatrix}$

11. $b \circ c$: $\begin{bmatrix}-1\\ 1\end{bmatrix} \circ \begin{bmatrix}1\\ 2\end{bmatrix} = \begin{bmatrix}-1\\ 2\end{bmatrix}$

12. b^Tb^T : Invalid, inner dimensions (2 and 1) doesn't match

13. $b + c^T$: (2 x 1) + (1 x 2) is invalid

14. $A^{-1}b$: $A^{-1} = -\frac{1}{2}\begin{bmatrix}4 & -2\\ -3 & 1\end{bmatrix} = \begin{bmatrix}-2 & 1\\ 1.5 & -0.5\end{bmatrix}$

$$A^{-1}b = \begin{bmatrix}-2 & 1\\ 1.5 & -0.5\end{bmatrix}\begin{bmatrix}-1\\ 1\end{bmatrix} \begin{bmatrix}3\\ -2\end{bmatrix}$$

15. $b^T A b = [2\ 2] \begin{bmatrix} -1 \\ 1 \end{bmatrix} = 0$

16. bAb^T : Invalid, because bA is invalid (#5)

3.2

In [14]:
```
import numpy as np

A = np.array([[1, 2], [3, 4]])
b = np.array([[-1], [1]])
c = np.array([[1], [2]])

print("1. AA:\n", A @ A)
print("---------")

print("2. AA^T:\n", A @ A.T)
print("---------")

print("3. Ab:\n", A @ b)
print("---------")

# 4. Ab^T (Invalid: (2x2) @ (1x2))
# print("\n4. Ab^T:\n", A @ b.T)
# Result: ValueError: shapes (2,2) and (1,2) not aligned

# 5. bA (Invalid: (2x1) @ (2x2))
# print("\n5. bA:\n", b @ A)
# Result: ValueError: shapes (2,1) and (2,2) not aligned

print("6. b^T A:\n", b.T @ A)
print("---------")

# 7. bb (Invalid: (2x1) @ (2x1))
# print("\n7. bb:\n", b @ b)
# Result: ValueError: shapes (2,1) and (2,1) not aligned

print("8. b^T b:\n", b.T @ b)
print("---------")

print("9. bb^T:\n", b @ b.T)
print("---------")

print("10. A * A:\n", A * A)
print("---------")

print("11. b * c:\n", b * c)
print("---------")

# 12. b^T b^T (Invalid: (1x2) @ (1x2))
# print("\n12. b^T b^T:\n", b.T @ b.T)
# Result: ValueError: shapes (1,2) and (1,2) not aligned

# 13. b + c^T
# Standard Linear Algebra: Invalid.
# Python Result: [[0, 1], [2, 3]]
# NumPy broadcasts the (2,1) and (1,2) shapes into a (2,2) matrix.

print("13. b + c^T :\n", b + c.T)
print("---------")

print("14. inv(A) b:\n", np.linalg.inv(A) @ b)
print("---------")

print("15. b^T A b:\n", b.T @ A @ b)
print("---------")
```

```
# 16. bAb^T (Invalid: (2x1) @ (2x2) is already invalid)
# print("\n16. bAb^T:\n", b @ A @ b.T)
# Result: ValueError: shapes (2,1) and (2,2) not aligned
```

```
1. AA:
 [[ 7 10]
 [15 22]]
---------
2. AA^T:
 [[ 5 11]
 [11 25]]
---------
3. Ab:
 [[1]
 [1]]
---------
6. b^T A:
 [[2 2]]
---------
8. b^T b:
 [[2]]
---------
9. bb^T:
 [[ 1 -1]
 [-1  1]]
---------
10. A * A:
 [[ 1  4]
 [ 9 16]]
---------
11. b * c:
 [[-1]
 [ 2]]
---------
13. b + c^T :
 [[0 1]
 [2 3]]
---------
14. inv(A) b:
 [[ 3.]
 [-2.]]
---------
15. b^T A b:
 [[0]]
---------
```

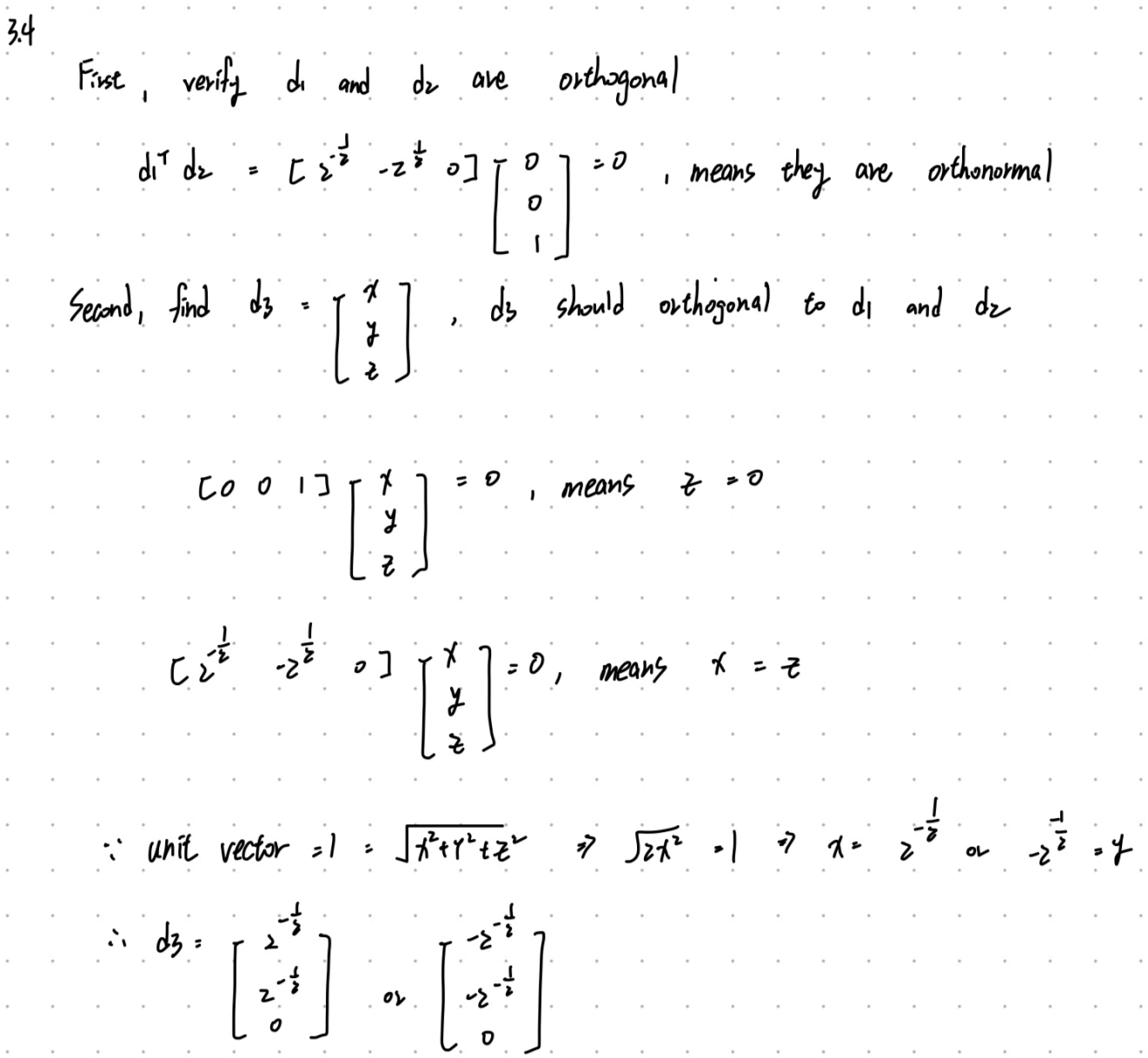
3.3

3.3

$$d_1 = \begin{bmatrix} 2^{-\frac{1}{2}} \\ -2^{-\frac{1}{2}} \\ 0 \end{bmatrix} \qquad \|d_1\| = \sqrt{\frac{1}{2} + \frac{1}{2} + 0} = 1$$

$$d_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \qquad \|d_2\| = \sqrt{1} = 1$$

3.4



3.5 - 1

In [15]:
```
B = np.array([[1, 2, 3], [2, 4, 5], [3, 5, 6]])

eigenvalues, eigenvectors = np.linalg.eig(B)
print("Eigenvalues: ", eigenvalues)
print("Eigenvectors: ", eigenvectors)
```

```
Eigenvalues:  [11.34481428 -0.51572947  0.17091519]
Eigenvectors:  [[-0.32798528 -0.73697623  0.59100905]
 [-0.59100905 -0.32798528 -0.73697623]
 [-0.73697623  0.59100905  0.32798528]]
```

3.5 - 2

Using the eigenvalue and eigenvectors of q1 to verify

In [16]:
```
v = eigenvectors[:, 0]
lam = eigenvalues[0]

B @ v, lam * v
```

Out[16]:
```
(array([-3.72093206, -6.70488789, -8.36085845]),
 array([-3.72093206, -6.70488789, -8.36085845]))
```

The output shows $\mathbf{Bv} = \lambda\mathbf{v}$

In [17]:
```
B @ (B @ v), (lam**2) * v
```

Out[17]:
```
(array([-42.2132832 , -76.06570795, -94.85238636]),
 array([-42.2132832 , -76.06570795, -94.85238636]))
```

This relations also extends to higher order $\mathbf{BBv} = \lambda^2\mathbf{v}$

3.5 - 3

In [18]:
```
dot_12 = np.dot(eigenvectors[:, 0], eigenvectors[:, 1])
dot_13 = np.dot(eigenvectors[:, 0], eigenvectors[:, 2])
dot_23 = np.dot(eigenvectors[:, 1], eigenvectors[:, 2])

print(f"v1 dot v2: {round(dot_12, 10)}")
print(f"v1 dot v3: {round(dot_13, 10)}")
print(f"v2 dot v3: {round(dot_23, 10)}")
```

```
v1 dot v2: -0.0
v1 dot v3: -0.0
v2 dot v3: -0.0
```

The inner products between distinct eigenvectors are approximately 0, confirming that the eigenvectors are orthogonal.
Because B is a real, symmetric matrix.

Exercise4 - Numerical Programming with Data

4.1

In [19]:
```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_excel("data/egrid2016.xlsx", header=1)
index = df['PLNGENAN'].idxmax()
most_energy = df.loc[index, ['PNAME', 'PLNGENAN']]
most_energy
```

Out[19]:
```
PNAME        Palo Verde
PLNGENAN    3.23775e+07
Name: 390, dtype: object
```

Palo Verde generated the most energy in 2016, producing about 32.4 million MWh.

4.2

In [20]:
```
idx_42 = df['PLCO2EQA'].idxmax()
plant_most_co2 = df.loc[idx_42, ['PNAME', 'PLCO2EQA']]

plant_most_co2
```

Out[20]:
```
PNAME       James H Miller Jr
PLCO2EQA           2.1725e+07
Name: 191, dtype: object
```

James H. Miller Jr produced the highest CO_2 emissions in 2016, emitting about 21.7 million tons of CO_2.

4.3

In [21]:
```
primary_fuel_most_co2 = df.loc[idx_42, 'PLPRMFL']
primary_fuel_most_co2
```

Out[21]:
```
'SUB'
```

The primary fuel of the plant with the highest CO2 emission is SUB.

4.4

In [22]:
```
idx_44 = df['LAT'].idxmax()
northernmost_plant = df.loc[idx_44, ['PNAME', 'LAT']]

northernmost_plant
```

Out[22]:
```
PNAME    Barrow
LAT      71.292
Name: 11, dtype: object
```

The northern-most power plant in the United States is Barrow

4.5

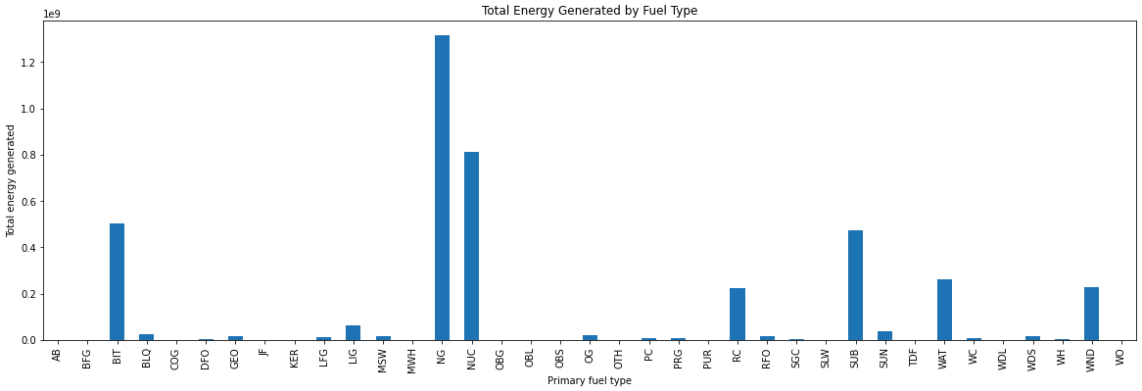
In [23]:
```
northernmost_state = df.loc[idx_44, 'PSTATABB']
northernmost_state
```

Out[23]:
```
'AK'
```

The northern-most power plant is located in Alaska

4.6

In [24]:
```
fuel_energy = df.groupby('PLPRMFL')['PLNGENAN'].sum()

plt.figure(figsize=(20, 6))
fuel_energy.plot(kind='bar')
plt.xlabel("Primary fuel type")
plt.ylabel("Total energy generated")
plt.title("Total Energy Generated by Fuel Type")
plt.show()
```



4.7

From the bar plot, natural gas(NG) produces the most energy in the United States.

4.8

In [25]:
```
hydro = df[df['PLPRMFL'] == 'WAT']
hydro_count = hydro['PSTATABB'].value_counts()
hydro_count.idxmax(), hydro_count.max()
```

Out[25]: ('CA', 264)

The state with the largest number of hydroelectric power plants is California, with 264 plants

4.9

In [26]:
```
coal_types = ['BIT', 'LTG', 'RC', 'SUB', 'WC']
coal_df = df[df['PLPRMFL'].isin(coal_types)]
coal_energy_by_state = coal_df.groupby('PSTATABB')['PLNGENAN'].sum
()
coal_energy_by_state.idxmax(), coal_energy_by_state.max()
```

Out[26]: ('TX', 80929850.0)

Texas has generated the most energy using coal

4.10

In [27]:
```
def fuel_group(fuel):
    coal = ['BIT', 'LIG', 'RC', 'SUB', 'WC']
    oil = ['DFO', 'JF', 'KER', 'RFO', 'WO']
    gas = ['BFG', 'COG', 'LFG', 'NG', 'OG', 'PG', 'PRG']
    renew = ['GEO', 'SUN', 'WAT', 'WDL', 'WDS', 'WND']
    if fuel in coal:
        return 'COAL'
    elif fuel in oil:
        return 'OIL'
    elif fuel in gas:
        return 'GAS'
    elif fuel in renew:
        return 'RENEW'
    else:
        return np.nan

df['FUEL_GROUP'] = df['PLPRMFL'].apply(fuel_group)
co2_by_fuel = df.groupby('FUEL_GROUP')['PLCO2EQA'].sum()
co2_by_fuel
```

Out[27]:
```
FUEL_GROUP
COAL     1.398669e+09
GAS      6.029717e+08
OIL      1.471201e+07
RENEW    3.421949e+06
Name: PLCO2EQA, dtype: float64
```

Coal produce the most C02 emission among the group in the United States.